Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №7

з дисципліни „ Професійний Python”

на тему „ Робота з базами даних у Python.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Мета роботи

Ознайомитися з організацією persistence layer у Python-застосунку та реалізувати збереження спортивних результатів у SQLite. У роботі використано SQL і DB-API, параметризовані запити, CRUD, транзакції, SQLAlchemy ORM, Repository Pattern, Alembic та інтеграційні тести на окремій базі даних.

2. Індивідуальний варіант і постановка задачі

Варіант 9 — облік спортивних результатів. Дані предметної області: вид спорту, спортсмен, спортивна категорія, числове значення результату та дані змагання. Необхідно зберегти попередній потоковий імпорт CSV із пакета sports_streaming і додати реляційне постійне сховище.

Реалізовано пов’язані таблиці sports, athletes і results; CRUD-репозиторії; фільтрацію результатів видом спорту; AVG та MAX статистики; ранжування через ORDER BY; параметризований DB-API запит; атомарне пакетне збереження з commit/rollback; міграцію дати змагання; інтеграційні тести на ізольованій SQLite.

3. Предметна область та структура проєкту

Один вид спорту містить багатьох спортсменів, а один спортсмен може мати багато результатів. Результат належить спортсмену, а його вид спорту визначається через зв’язки Result → Athlete → Sport. Імпортована CSV-обробка залишається початковою точкою застосунку; прапорець --persist-db додатково записує валідні записи у БД.

```
sports_results_lab07/
├── pyproject.toml, README.md, alembic.ini
├── migrations/
│   ├── env.py
│   └── versions/  (initial_schema, add_competition_date)
├── src/sports_streaming/
│   ├── main.py, models.py, services.py, validators.py, readers.py
│   ├── database.py, db_models.py, repositories.py
│   └── persistence_service.py
└── tests/
    ├── conftest.py
    └── integration/ (test_persistence.py, test_migrations.py)
```

4. Схема бази даних



Рисунок 1 — ER-схема та зв’язки один-до-багатьох

Таблиц	Поля та обмеження

sports	id INTEGER PRIMARY KEY; name VARCHAR(80) NOT NULL UNIQUE; discipline VARCHAR(80) NOT NULL.
athletes	id INTEGER PRIMARY KEY; name VARCHAR(120) NOT NULL; category VARCHAR(80) NOT NULL; sport_id INTEGER NOT NULL REFERENCES sports(id) ON DELETE CASCADE; UNIQUE(sport_id, name).
results	id INTEGER PRIMARY KEY; athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE; value FLOAT NOT NULL CHECK(value >= 0); competition_name VARCHAR(120) NOT NULL; competition_date DATE NULL (додано міграцією).

Первинні ключі id однозначно ідентифікують рядки. Зовнішні ключі sport_id та athlete_id забезпечують зв’язки між таблицями. SQLite foreign key enforcement вмикається через PRAGMA foreign_keys=ON. UNIQUE не допускає дублювання назви спортсмена в межах одного виду спорту, а CHECK забороняє від’ємне значення результату.

4.1 SQL для створення таблиць

```
CREATE TABLE sports (
  id INTEGER PRIMARY KEY,
  name VARCHAR(80) NOT NULL UNIQUE,
  discipline VARCHAR(80) NOT NULL
);
CREATE TABLE athletes (
  id INTEGER PRIMARY KEY, name VARCHAR(120) NOT NULL,
  category VARCHAR(80) NOT NULL, sport_id INTEGER NOT NULL,
  FOREIGN KEY (sport_id) REFERENCES sports(id) ON DELETE CASCADE,
  UNIQUE (sport_id, name)
);
CREATE TABLE results (
  id INTEGER PRIMARY KEY, athlete_id INTEGER NOT NULL,
  value FLOAT NOT NULL CHECK (value >= 0),
  competition_name VARCHAR(120) NOT NULL, competition_date DATE,
  FOREIGN KEY (athlete_id) REFERENCES athletes(id) ON DELETE CASCADE
);
```

5. Persistence layer та ORM

database.py створює Engine і Session factory. Для SQLite на кожному з’єднанні вмикається контроль зовнішніх ключів. db_models.py описує таблиці сучасним декларативним SQLAlchemy API, а relationship() дає навігацію між пов’язаними об’єктами.

```
def make_engine(url: str = "sqlite:///sports_results.db", *, echo: bool = False) -> Engine:
    engine = create_engine(url, echo=echo)
    if url.startswith("sqlite"):
        @event.listens_for(engine, "connect")
        def enable_foreign_keys(dbapi_connection, _record):
            cursor = dbapi_connection.cursor()
            cursor.execute("PRAGMA foreign_keys=ON")
            cursor.close()
    return engine

def make_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, expire_on_commit=False)
class Athlete(Base):
    __tablename__ = "athletes"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False)
    category: Mapped[str] = mapped_column(String(80), nullable=False)
    sport_id: Mapped[int] = mapped_column(ForeignKey("sports.id", ondelete="CASCADE"))
    sport: Mapped[Sport] = relationship(back_populates="athletes")
    results: Mapped[list[Result]] = relationship(back_populates="athlete", cascade="all, delete-orphan")
```

6. Repository Pattern і CRUD

SportRepository, AthleteRepository та ResultRepository приховують деталі SQLAlchemy від сервісного коду. Усі репозиторії мають операції додавання, читання, оновлення та видалення. Сервіс пакетного імпорту координує реєстрацію пов'язаних довідникових записів і атомарне збереження результатів.

```
sport = SportRepository(session).add("Swimming", "Aquatics")
athlete = AthleteRepository(session).add("Anna Koval", "Junior", sport.id)
result = ResultRepository(session).add(athlete.id, 52.4, "City Cup")
repo.update(result.id, value=51.9)  # Update
repo.get(result.id)               # Read
repo.delete(result.id)            # Delete
```

6.1 Параметризований пошук через DB-API

Функція filter_results_dbapi використовує sqlite cursor і placeholder ?, тому значення фільтра передається окремо від тексту SQL. Це захищає пошук від SQL injection. Інтеграційний тест також перевіряє рядок-атаку і отримує порожній результат.

```
cursor.execute("""SELECT r.id, a.name, s.name, r.value
  FROM results r JOIN athletes a ON a.id=r.athlete_id
  JOIN sports s ON s.id=a.sport_id WHERE s.name = ?
  ORDER BY r.value ASC""", (sport,))
```

6.2 Статистика і рейтинг

ResultRepository.average використовує SQL AVG(value), maximum — SQL MAX(value), a ranking формує впорядкований список через ORDER BY. У попередньому проєкті часові дисципліни трактувалися як "менше — швидше", тому best зберігає цю сумісну поведінку й сортує за зростанням. maximum надає MAX для дисциплін, де більший бал або дистанція є кращими; напрям ранжування залежить від метрики спорту.

7. Транзакції, commit, rollback і ACID

save_result_batch перевіряє кожен запис і додає всі результати в межах однієї операції. Після успішного flush викликається commit. Якщо невідомий спортсмен або некоректне значення перериває пакет, викликається rollback і жоден результат із цього пакета не лишається в БД.

```
try:
    for record in records:
        if record.result < 0: raise ValueError("negative result")
        session.add(Result(athlete_id=athlete_id, value=record.result, ...))
    session.flush()
    session.commit()
except Exception:
    session.rollback()
    raise
```

ACID	Як забезпечується
Atomicity	Пакет додається цілком або повністю скасовується session.rollback().
Consistency	PK/FK, UNIQUE та CHECK обмеження зберігають коректність зв'язків і значень.
Isolation	SQLite та окремі Session ізолюють транзакційний контекст клієнтських операцій.
Durability	Після commit SQLite фіксує зміну у файлі БД.

8. Міграції Alembic

Версія 0001_initial створює sports, athletes і results без дати змагання. Наступна ревізія 0002_competition_date додає поле competition_date до results; downgrade видаляє його. Перевірка запускає міграції на тимчасовій файловій БД і перевіряє таблиці та колонку.

```
def upgrade() -> None:
    op.add_column("results", sa.Column("competition_date", sa.Date(), nullable=True))

def downgrade() -> None:
    op.drop_column("results", "competition_date")
python -m alembic upgrade head
python -m alembic current
```

9. Integration tests і результати запуску

Тестова fixture створює окрему SQLite in-memory базу та вмикає foreign keys. Перевіряються CRUD для Sport, Athlete та Result; relationship athlete-sport; FK/CHECK обмеження; SQL DB-API фільтр; AVG, MAX та рейтинг; повний commit пакетної вставки; rollback при помилці; міграція Alembic.

```
python -m pytest -v
========================= 10 passed in 0.52s =========================
```

```
(.venv) PS C:\Users\Veronika\Phyton\lab7> python -m pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Veronika\Phyton\lab7\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Veronika\Phyton\lab7
configfile: pyproject.toml
testpaths: tests
collected 10 items

tests/integration/test_migrations.py::test_alembic_applies_schema_and_competition_date PASSED                [ 10%]
tests/integration/test_persistence.py::test_sport_crud PASSED                                                [ 20%]
tests/integration/test_persistence.py::test_athlete_crud PASSED                                              [ 30%]
tests/integration/test_persistence.py::test_result_crud PASSED                                               [ 40%]
tests/integration/test_persistence.py::test_foreign_key_and_check_constraints PASSED                         [ 50%]
tests/integration/test_persistence.py::test_parameterized_dbapi_filter_and_sport_isolation PASSED            [ 60%]
tests/integration/test_persistence.py::test_best_average_and_ranking PASSED                                  [ 70%]
tests/integration/test_persistence.py::test_batch_transaction_commit PASSED                                  [ 80%]
tests/integration/test_persistence.py::test_batch_transaction_rolls_back_all_on_invalid_row PASSED           [ 90%]
tests/integration/test_persistence.py::test_persist_imported_results_creates_related_records PASSED          [100%]

============================================= 10 passed in 0.55s ==============================================
```

Рисунок 2 — Фактичний результат виконання pytest -v

```
Запуск імпорту CSV із збереженням у SQLite

2026-10-28 17:14:30,778 | WARNING | sports_streaming.services | Invalid record at line 6 (result): Result
   must be numeric
2026-10-28 17:14:30,778 | WARNING | sports_streaming.services | Invalid record at line 7 (athlete):
   Athlete name is required
2026-10-28 17:14:30,778 | WARNING | sports_streaming.services | Invalid record at line 8 (category):
   Category is not allowed
2026-10-28 17:14:30,837 | INFO | __main__ | Persisted 5 results in SQLite
2026-10-28 17:14:30,838 | INFO | sports_streaming.services | Sports results import/export started
2026-10-28 17:14:30,844 | INFO | sports_streaming.services | Sports results import/export completed in
   0.0062 s
2026-10-28 17:14:30,845 | INFO | __main__ | Total=8 valid=5 invalid=3 exported=5
2026-10-28 17:14:30,845 | INFO | __main__ | cycling: count=1 average=42.8000 best numeric record=Iryna
   Bondar (42.8000)
2026-10-28 17:14:30,845 | INFO | __main__ | running: count=3 average=12.4333 best numeric record=Petro
   Novak (11.9000)
2026-10-28 17:14:30,845 | INFO | __main__ | swimming: count=1 average=58.2000 best numeric record=Maksym
   Shevchenko (58.2000)
```

Рисунок 3 — Фактичний запуск CSV імпорту із записом до SQLite

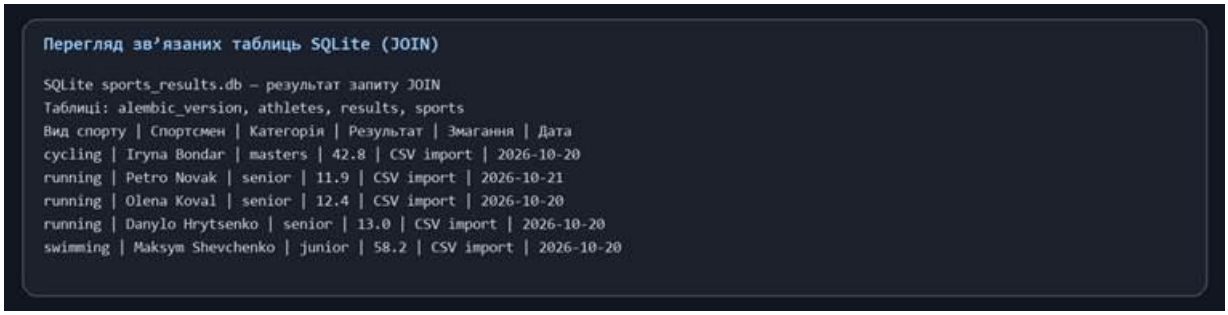


Рисунок 4 — Результат SQL JOIN за таблицями sports, athletes і results

9.1 Команди встановлення та запуску

```
python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install -e ".[test]"

python -m alembic upgrade head

python -m sports_streaming.main --persist-db

python -m pytest -v
```

Пробний імпорт містив 8 рядків: 5 валідних рядків було експортовано та збережено, 3 некоректні рядки потрапили у журнал як помилки tolerant режиму. Фактичний результат тестів: 10 passed. Тестова БД не є production sports_results.db.

10. Відповіді на контрольні запитання

1. Що таке база даних? Організоване сховище даних для пошуку, додавання, зміни й видалення інформації.

2. Що таке реляційна база даних? База, у якій дані зберігаються у таблицях, пов’язаних ключами.

3. Що таке таблиця? Структура з рядків і стовпців для записів одного типу.

4. Що таке рядок таблиці? Один запис або екземпляр сутності.

5. Що таке поле? Стовпець, що зберігає одну характеристику кожного запису.

6. Що таке SQL? Мова визначення, читання та зміни даних у реляційних БД.

7. Для чого використовується CREATE TABLE? Для створення таблиці та опису колонок і обмежень.

8. Що означає CRUD? Create, Read, Update, Delete — створити, прочитати, змінити, видалити.

9. Яка SQL-команда відповідає Create? INSERT додає новий запис.

10. Яка команда відповідає Read? SELECT читає записи.

11. Яка команда відповідає Update? UPDATE змінює записи.

12. Яка команда відповідає Delete? DELETE видаляє записи.

13. Що таке SQLite? Вбудована реляційна СУБД, що зберігає БД у файлі та не потребує окремого сервера.

14. Які переваги SQLite? Просте налаштування, переносимість, малий розмір і наявність стандартного модуля sqlite3 у Python.

15. Що таке Python DB-API? Стандартний інтерфейс Python для роботи з реляційними СУБД.

16. Для чого використовується Connection? Для відкриття сесії доступу до БД та керування транзакціями.

17. Для чого використовується Cursor? Для виконання SQL-запитів і читання їх результатів.

18. Для чого потрібен commit()? Щоб зафіксувати зміни транзакції.

19. Для чого потрібен rollback()? Щоб скасувати незакомічені зміни у разі помилки.

20. Що таке SQL injection? Атака, у якій введене значення змінює структуру SQL-запиту.

21. Що таке parameterized query? Запит із placeholders, де параметри передаються окремо від SQL-тексту.

22. Чому не рекомендується формувати SQL через f-string? Бо введення може інтерпретуватися як код SQL і спричинити injection.

23. Що таке Primary Key? Колонка або набір колонок, що унікально ідентифікує рядок.

24. Що таке Foreign Key? Обмеження, яке посилається на ключ іншої таблиці та підтримує цілісність зв'язку.

25. Що таке транзакція? Група змін, яку БД фіксує як єдину операцію або скасовує повністю.

26. Що означає Atomicity? Або виконуються всі кроки транзакції, або жоден.

27. Що означає Consistency? Транзакція переводить БД з одного коректного стану в інший.

28. Що означає Isolation? Паралельні транзакції не повинні бачити некоректні проміжні зміни одна одної.

29. Що означає Durability? Після commit зміни залишаються збереженими.

30. Що означає ACID? Atomicity, Consistency, Isolation, Durability — властивості надійних транзакцій.

31. Що таке persistence layer? Шар програми, який відповідає за збереження й отримання даних.

32. Для чого потрібний Repository Pattern? Щоб надати сервісам зрозумілі методи даних та приховати конкретні SQL/ORM операції.

33. Чому Repository відокремлюють від Service Layer? Щоб бізнес-правила не залежали від способу зберігання, а шари було простіше тестувати.

34. Що таке ORM? Технологія відповідності між об'єктами Python і таблицями/рядками БД.

35. Для чого використовується SQLAlchemy? Для побудови SQL-запитів, ORM-моделей, сесій і транзакцій у Python.

36. Що таке ORM model? Python-клас, зіставлений із таблицею, поля якого відповідають її колонкам.

37. Що таке relationship()? Декларація навігаційного зв'язку між ORM-моделями.

38. Що означає one-to-many? Один запис батьківської таблиці може мати багато пов'язаних дочірніх записів.

39. Що означає many-to-many? Багато записів з обох таблиць пов'язані між собою через таблицю-зв'язку.

40. Що таке migration? Версійований опис зміни схеми БД із кроком застосування та зазвичай відкату.

41. Для чого використовується Alembic? Для керування міграціями SQLAlchemy-схеми.

42. Для чого потрібна команда alembic upgrade head? Щоб послідовно застосувати всі міграції до найновішої ревізії.

43. Що таке integration test? Тест, який перевіряє взаємодію кількох компонентів, тут — репозиторію й справжньої SQLite.

44. Чим integration test відрізняється від unit test? Unit test ізолює одну функцію; integration test перевіряє взаємодію зі сховищем та схемою.

45. Навіщо використовувати окрему test database? Щоб тестові зміни не впливали на робочі дані та тести були відтворюваними.

46. Що перевіряє тест rollback? Що після помилки частково записані дані відсутні в БД.

47. Для чого використовуються fixtures? Для повторного створення контрольованих передумов і очищення ресурсів тесту.

48. Що робить SQL JOIN? Об’єднує рядки таблиць відповідно до умови зв’язку між ключами.

49. Що робить GROUP BY? Групує рядки за значенням колонок, зазвичай для агрегатних функцій.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX? COUNT рахує рядки, SUM підсумовує, AVG обчислює середнє, MIN/MAX знаходять крайні значення.

11. Висновок

У лабораторній роботі persistence layer спортивних результатів інтегровано з попереднім CSV-проєктом. SQLite схема має три пов’язані таблиці з PK/FK та перевірками; SQLAlchemy моделі і Repository забезпечують CRUD; параметризований DB-API пошук і статистичні запити працюють із реляційними даними. Пакетне додавання використовує commit або rollback, Alembic міграція додає дату змагання, а 10 integration tests підтверджують CRUD, зв’язки, фільтрацію, агрегати та міграції на ізольованій тестовій БД.